

A Decomposition Approach for Optimal Building Design on Irregular Urban Lots

An Operations Research Formulation

LandValue Technical Report

February 27, 2026

Abstract

We present a mathematical optimization framework for the automated design of residential buildings on irregular urban lots. In urban real estate development, determining the most profitable building that can be legally constructed on a given lot requires navigating a complex interplay of geometric constraints imposed by zoning regulations, volumetric restrictions arising from shadow projection and inclined-plane rules, and market-driven apartment mix preferences. This problem combines continuous geometric decisions (footprint shape, setback distances), discrete combinatorial decisions (number and type of apartments per floor), and non-convex constraints (shadow cones, rasante planes), making a monolithic optimization approach computationally intractable.

We decompose the problem into three stages: (A) a *buildable envelope computation* that reduces the full set of three-dimensional zoning constraints to a discrete height map over the lot polygon, (B) a *floor plan optimization* that combines corridor routing on the medial axis of irregular floor footprints with one-dimensional knapsack-based apartment packing along the corridor strips, and (C) a *multi-floor assembly* that enforces building-wide constraints such as constructibility limits, density caps, and parking requirements. The approach handles rectangular, L-shaped, H-shaped, U-shaped, and T-shaped floor plans within a unified framework. We analyze the computational complexity of each stage and show that the full building optimization solves in sub-second time for typical urban lots, enabling batch evaluation of thousands of development scenarios.

Contents

1	Introduction and Problem Statement	3
2	Problem Decomposition	4
3	Stage A: Buildable Envelope Computation	6
3.1	Discretization	6
3.2	Facade Setback as Decision Variable	6
3.3	Parametric Height Map	7
3.3.1	Maximum Height (Distance-Dependent)	8
3.3.2	Setbacks (Lateral and Rear)	8
3.3.3	Rasante (Inclined Plane)	9
3.3.4	Shadow Projection	9
3.4	Optimal Facade Setback Selection	10
3.5	Floor Footprint Extraction	11
3.6	Ground Occupation Constraint	12

4	Stage B: Floor Plan Optimization	12
4.1	Stage B.1: Skeleton Graph and Corridor Routing	13
4.1.1	Rectilinear Medial Axis	13
4.1.2	Corridor as Connected Subtree	13
4.2	Stage B.2: Strip Decomposition	14
4.2.1	Strip Segments	14
4.2.2	Junction Zones	15
4.3	Stage B.3: Apartment Template Packing	16
4.3.1	Apartment Templates	16
4.3.2	Bounded Knapsack Formulation	17
4.3.3	Market Mix via Lagrangian Relaxation	18
4.4	Stage B.4: Joint Floor Optimization	18
5	Stage C: Multi-floor Assembly	19
5.1	Floor Grouping	20
5.2	Building-wide Constraints	20
5.3	Parking Requirements	20
5.3.1	Underground Parking Optimization	21
5.4	Total Building Value	21
6	Computational Complexity	22
7	Conclusion	23

1 Introduction and Problem Statement

The design of a residential building on an urban lot is fundamentally an optimization problem. A real estate developer seeks to maximize the economic value of the project—the total revenue from apartment sales minus construction and land costs—subject to a dense web of regulatory constraints that govern what can be built on any given parcel. In practice, this design process is carried out manually by architects working iteratively with feasibility analysts, a process that is both time-consuming and inherently incomplete: only a handful of design variants are explored out of a vast space of possibilities.

Consider an urban lot defined by a polygon $P \subset \mathbb{R}^2$ on which a residential building is to be constructed. The lot sits within a municipality that has adopted a *Plan Regulador Comunal* (PRC), a zoning instrument that assigns each lot to a zone with specific building regulations. The lot is subject to a set of urban planning regulations $\mathcal{R}$ that constrain the building geometry (setbacks, maximum height, shadow projections, floor-area ratios) and a set of market conditions $\mathcal{P}$ that determine the economic value of different apartment types. The regulations are layered: national codes provide baseline rules, while municipal plans may impose stricter or additional constraints.

Definition 1 (Building Design Problem). *Given a lot polygon P , a regulation set $\mathcal{R}$, and market parameters $\mathcal{P}$, find a building design $\mathcal{B}$ that maximizes total economic value while satisfying all regulatory constraints. A building design $\mathcal{B}$ consists of:*

- (i) *A set of floor footprints $\{F_z\}_{z=1}^Z$, where $F_z \subseteq P$ is the buildable area at floor level z ,*
- (ii) *A corridor layout $C_z \subset F_z$ for each floor,*
- (iii) *An assignment of apartment templates to positions on each floor.*

The regulatory framework we consider is based on the Chilean *Ordenanza General de Urbanismo y Construcciones* (OGUC), a national building code that governs all urban development in Chile. While the formulation is presented in this regulatory context, the mathematical structure is general and applies to any jurisdiction with analogous volumetric and density controls. The specific constraints include:

- **Setbacks** (*antepared, distanciamiento*): minimum distances from lot boundaries. The *antepared* is a mandatory front setback from the street, typically 3–5 m, intended to create a uniform building line along the block. The *distanciamiento* is a lateral and rear separation from neighboring lots that increases with building height, designed to ensure privacy, ventilation, and emergency access between buildings.
- **Height limits**: absolute maximum building height in meters or floors, set by the municipal zoning plan. These vary dramatically across zones—from 2 floors in low-density residential areas to 20+ floors in high-density mixed-use corridors.
- **Rasante**: an inclined plane rising from the lot boundary at a regulated angle (commonly 70° from horizontal in Chile). This constraint creates a tapered building envelope that prevents tall buildings from creating canyon-like conditions on narrow streets. The building volume must lie entirely below all applicable rasante planes.
- **Shadow projections**: limits on the shadow cast by the building onto public spaces (streets, plazas, parks) during regulated hours, typically at the winter solstice and equinoxes. This constraint is particularly binding for tall buildings on north-facing lot boundaries (in the southern hemisphere).
- **Constructibility coefficient** (c_{con}): maximum ratio of total built area (summed across all floors) to lot area. A coefficient of 3.0, for example, means that a 1000 m² lot can have at most 3000 m² of total floor area.
- **Ground occupation coefficient** (c_{occ}): maximum ratio of the ground floor footprint area to the lot area. This ensures that a minimum fraction of the lot remains as open space at ground level for landscaping, access, and permeability.

- **Density:** maximum number of dwelling units per hectare, controlling population density. Chilean regulations under DFL 2 also provide tax benefits for apartments below certain size thresholds, creating economic incentives that interact with the density constraint.

The problem is computationally challenging because it combines three fundamentally different types of decisions: (i) *continuous geometric decisions* (footprint shape, position, and rotation within the lot), (ii) *discrete combinatorial decisions* (number of apartments, their types and sizes, and their arrangement on each floor), and (iii) *non-convex constraints* (shadow projections depend on building height in a non-convex manner; rasante planes create polyhedral but non-convex feasible volumes; the interaction between footprint shape and apartment packing efficiency is highly non-linear).

A monolithic mixed-integer programming approach that attempts to jointly optimize all decisions would require binary variables for every possible apartment placement on a 2D grid at every floor level, easily reaching 10^6 binary variables for a moderately sized lot. We address this through a three-stage decomposition that separates the geometric envelope computation from the combinatorial apartment packing, exploiting the natural structure of the problem to achieve exact or near-exact solutions in sub-second computation time.

2 Problem Decomposition

The key insight driving our approach is that the building design problem has a natural hierarchical structure. The geometric regulations that determine *where* and *how high* the building can rise depend on the lot shape, its surroundings (streets, neighboring buildings), the zoning parameters, and one critical design decision: how far back the building is set from the street. This *facade setback distance* is a continuous decision variable that creates a fundamental trade-off: pulling the building further from the street increases the allowable height (and thus the number of floors) but reduces the available footprint area. Crucially, once the facade setback is fixed, the buildable envelope becomes fully determined by the lot geometry and regulations, independent of how apartments are arranged within the volume. Conversely, the apartment packing problem on a given floor depends only on the shape and dimensions of that floor’s footprint, not on the regulatory details that produced it. This separation of concerns motivates the following decomposition:

Stage A — Envelope Computation (Section 3): Starting from the raw lot polygon and the full set of applicable regulations, this stage computes a *family of buildable envelopes*, each parameterized by the facade setback distance d_f from the street. For each candidate setback, a discrete *height map* is computed: a two-dimensional grid over the lot where each cell stores the maximum number of floors that can be built at that location, considering all applicable setbacks, rasante planes, shadow restrictions, and the height limit corresponding to that facade distance. The optimal setback is selected by evaluating the downstream building value for each candidate, capturing the trade-off between footprint area and allowable height. This effectively transforms a complex three-dimensional geometric feasibility problem into a one-dimensional search over facade positions, each yielding a compact two-dimensional data structure that encapsulates all regulatory constraints.

Stage B — Floor Plan Optimization (Section 4): For each distinct floor footprint extracted from the height map, this stage determines the optimal internal layout: where to place the corridor, where to locate the circulation core (stairs and elevator), and how to distribute apartments along the corridor to maximize revenue. This stage is itself decomposed into four sub-stages that exploit the geometric structure of corridor-based residential buildings:

- B.1 *Skeleton graph construction and corridor routing:*** the medial axis of the floor polygon provides a natural corridor backbone, and valid corridors are enumerated as connected subtrees.

- B.2 Strip decomposition:** each corridor edge defines two strips of usable area (one on each side), reducing the two-dimensional floor plan to a collection of one-dimensional packing problems.
- B.3 Apartment packing via bounded knapsack:** each strip is packed with apartment templates using dynamic programming on a one-dimensional bounded knapsack formulation.
- B.4 Joint optimization:** corridor topologies, junction treatments, and strip solutions are combined by exhaustive enumeration over the (small) space of structural options.

Stage C — Multi-floor Assembly (Section 5): The optimized floor plans are assembled into a complete building, and building-wide constraints are enforced: total constructibility (the sum of all floor areas must not exceed the regulatory limit), maximum density (total dwelling units per hectare), parking requirements (computed from the apartment mix), and underground parking level dimensioning.

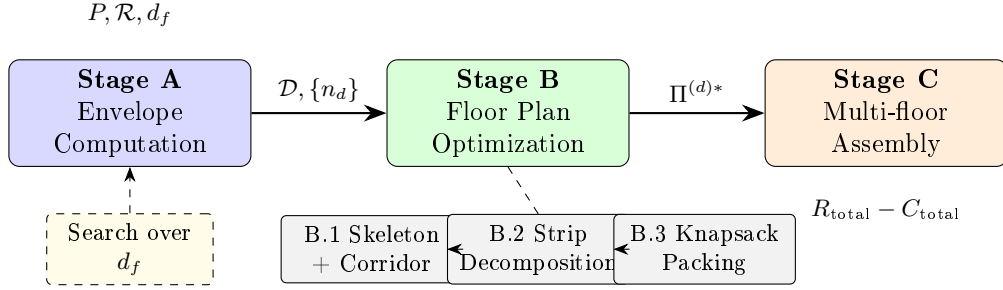


Figure 1: Three-stage decomposition pipeline. Stage A computes the buildable envelope for each facade setback candidate d_f , producing the set of distinct footprints $\mathcal{D}$ with floor counts $\{n_d\}$. Stage B optimizes the floor plan for each distinct footprint independently, through skeleton construction, strip decomposition, and knapsack packing. Stage C assembles the building and evaluates the total profit $R_{\text{total}} - C_{\text{total}}$.

Remark 1 (Decomposition Justification). *The decomposition is valid under four structural observations:*

- Once the facade setback distance d_f is fixed, the buildable envelope depends only on the lot geometry and regulations, not on the apartment layout. No interior design decision can change the rasante plane, the shadow cone, or the height limit—these are fully determined by d_f and the lot context.
- The facade setback d_f is a one-dimensional continuous variable that can be efficiently searched by discretization (at δ -meter steps) or golden-section search, since the total building value as a function of d_f is typically unimodal (increasing returns from height eventually overtaken by decreasing returns from shrinking footprint).
- Floor plans at different levels are coupled only through building-wide aggregate constraints (total area, total units, structural alignment), not through geometric interactions between floors. Once the footprint at each level is known, the interior layout of floor 5 does not affect what is feasible on floor 8.
- Floors with identical footprints admit identical optimal layouts (assuming the same market conditions apply), enabling a single floor plan to be replicated across multiple levels. This is also consistent with structural engineering practice, where column grids are typically maintained vertically for load transfer.

Example 1 (Decomposition in Practice). *Consider a trapezoidal corner lot of $2\,000\text{ m}^2$ in a zone where the maximum height is a function of the building-to-street distance, with a constructibility coefficient of 4.5. Stage A evaluates several facade setback candidates: at the mandatory minimum setback of 5 m, the height regulation allows 8 floors; at 8 m setback, it allows 11 floors; at 10 m, it allows the zoning maximum of 12 floors. For each candidate, the full height map is computed—the rasante from the narrow street side limits the first 3 m band to 6 floors, while the interior reaches the height limit. Stage A selects the 8 m setback as optimal: it produces 3 distinct footprints (floors 1–6 as a large L-shape, floors 7–9 as a reduced L, floors 10–11 as a compact rectangle), yielding higher total value than the 5 m setback (fewer floors on a larger footprint) or the 10 m setback (more floors on a significantly smaller footprint). Stage B solves three independent floor plan problems. Stage C verifies the constructibility constraint and computes parking for the resulting 58 apartments.*

3 Stage A: Buildable Envelope Computation

The first stage of the optimization determines the three-dimensional volume within which the building may legally exist. Unlike a naive approach that would treat this volume as fixed, our formulation recognizes a critical coupling: the maximum allowable building height is a function of how far the building facade is set back from the street. A building placed at the mandatory minimum setback may be limited to, say, 8 floors; the same building pulled 5 meters further from the street may be allowed 12 floors. This creates a fundamental trade-off between *footprint area* (which decreases as the building retreats from the street) and *building height* (which increases), and the optimal balance depends on the specific lot geometry, regulations, and market conditions.

To handle this coupling, we formulate Stage A as a *parametric envelope computation*: for each candidate facade setback distance d_f , we compute a complete height map encoding all geometric regulations, then select the setback that maximizes total building value. This transforms the coupled 3D geometric problem into a one-dimensional search over facade positions, each yielding a self-contained 2D height map.

3.1 Discretization

To make the height map computationally tractable, we discretize the lot polygon onto a regular grid. The choice of resolution δ balances accuracy against computation: finer grids capture boundary details more precisely but increase computation proportionally. A resolution of $\delta = 0.5\text{ m}$ strikes a practical balance, as it is finer than typical construction tolerances while keeping the grid size manageable (a $2\,000\text{ m}^2$ lot produces roughly 8 000 grid cells).

Let the lot polygon P be embedded in a coordinate system aligned with the UTM projection (Universal Transverse Mercator, zone 19S for central Chile). We discretize P into a regular grid:

$$G = \{(i, j) \in \mathbb{Z}^2 : (i\delta, j\delta) \in P\}.$$

3.2 Facade Setback as Decision Variable

Let $\partial P_{\text{street}} = \{\ell_k : k \in K_{\text{street}}\}$ denote the subset of lot boundary segments that face a street, and let $d_f^{\min}$ be the mandatory minimum front setback (*antepatio*). The *facade setback distance* $d_f \geq d_f^{\min}$ is the perpendicular distance from the street-facing boundary to the building facade line. This distance is a decision variable that determines two key quantities:

- (i) **Maximum building height.** The regulation specifies the maximum height as a function of the building-to-street distance:

$$h_{\max}(d_f) = f(d_f), \tag{1}$$

where f is a regulatory function (typically piecewise-linear or affine). For example, a common formulation is $h_{\max}(d_f) = \beta \cdot (d_f + w_{\text{street}})$, where w_{street} is the street width and β is a coefficient set by the zoning plan (e.g., $\beta = 1.5$ means the building height may be up to 1.5 times the distance from the building facade to the opposite side of the street). Other zones may use step functions, logarithmic formulas, or lookup tables.

- (ii) **Buildable footprint area.** Cells between the street boundary and the facade line at distance d_f cannot be built upon (they form the front garden / setback zone). The set of cells available for construction at ground level is:

$$G(d_f) = \{(i, j) \in G : \text{dist}(i, j; \ell_k) \geq d_f, \forall k \in K_{\text{street}}\}. \quad (2)$$

As d_f increases, $G(d_f)$ shrinks—the building loses frontage area.

The trade-off is clear: increasing d_f allows a taller building (more floors, more total sellable area per unit of footprint) but on a smaller footprint (fewer apartments per floor). The optimal d_f depends on the interaction between the height-distance function, the lot depth, the *rasante*, and the apartment packing efficiency at different floor dimensions.

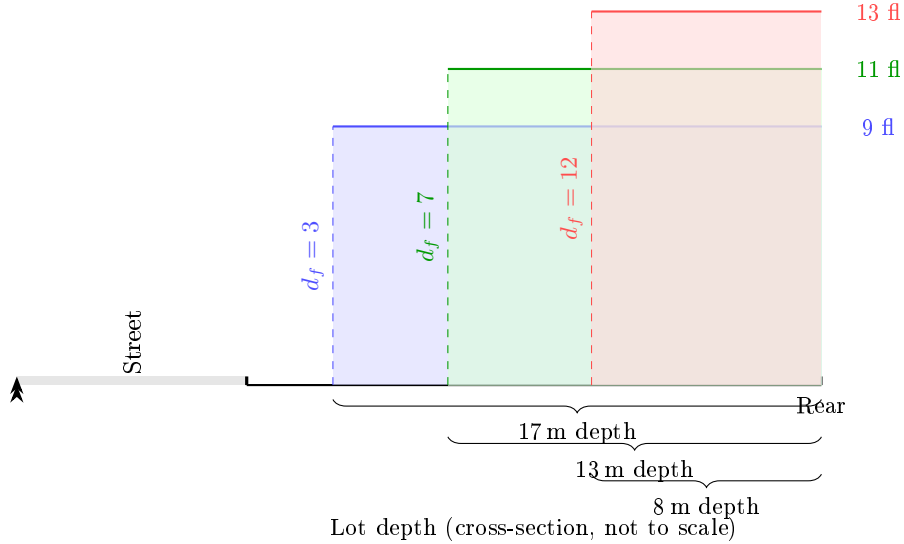


Figure 2: Facade setback trade-off (cross-section view). Three candidate setback distances are shown for a 20 m-deep lot: $d_f = 3$ m yields 9 floors on 17 m of depth (blue), $d_f = 7$ m yields 11 floors on 13 m (green), and $d_f = 12$ m yields 13 floors on only 8 m (red). The optimal d_f balances height gain against footprint loss.

3.3 Parametric Height Map

For a given facade setback d_f , we define the height map $H(\cdot; d_f)$ over the buildable cells $G(d_f)$.

Definition 2 (Parametric Height Map). *The parametric height map is a function $H : G(d_f) \times \mathbb{R}_+ \rightarrow \mathbb{N}_0$ where $H(i, j; d_f)$ represents the maximum number of floors that can be built at grid cell (i, j) when the building facade is set back at distance d_f from the street. A cell with $H(i, j; d_f) = 0$ lies within a setback zone (lateral or rear) and cannot be built upon at any height.*

Each regulation $r \in \mathcal{R}$ imposes an upper bound $H_r(i, j; d_f)$ on the buildable height at cell (i, j) under facade setback d_f . Since the building must comply with *all* regulations simultaneously, the final height map is obtained by taking the pointwise minimum:

$$H(i, j; d_f) = \min_{r \in \mathcal{R}} H_r(i, j; d_f), \quad \forall (i, j) \in G(d_f). \quad (3)$$

This formulation preserves the important computational advantage of independent evaluation: each regulation can be computed independently (and potentially in parallel), and the results are combined through a simple elementwise minimum. Adding a new regulation requires only implementing its height bound function H_r ; the rest of the pipeline remains unchanged.

We now define each regulatory constraint formally.

3.3.1 Maximum Height (Distance-Dependent)

Unlike a simple zoning cap, the maximum height constraint depends on the facade setback distance d_f . The regulation specifies how tall the building may be as a function of its distance from the street, creating the central trade-off of Stage A.

Let $h_{\max}(d_f)$ be the regulatory maximum building height (in meters) as given by (1), and h_f the floor-to-floor height (typically 2.7–3.0 m for residential buildings, and up to 4.0 m for the ground floor if commercial use is planned). The height limit applies uniformly to all cells for a given facade position:

$$H_{\text{height}}(i, j; d_f) = \left\lfloor \frac{h_{\max}(d_f)}{h_f} \right\rfloor, \quad \forall (i, j) \in G(d_f). \quad (4)$$

Note that while this bound is spatially uniform (all cells share the same cap for a given d_f), it varies across different facade setback candidates. A setback of $d_f = 5$ m might yield a cap of 8 floors, while $d_f = 10$ m might yield 12 floors.

Example 2 (Height-Distance Trade-off). *Consider a rectangular lot 20 m deep (from street to rear boundary) with mandatory minimum setback $d_f^{\min} = 3$ m, street width $w_{\text{street}} = 15$ m, and $\beta = 1.5$. At $d_f = 3$ m: $h_{\max} = 1.5 \times (3 + 15) = 27$ m ≈ 9 floors, with 17 m of usable depth. At $d_f = 7$ m: $h_{\max} = 1.5 \times (7 + 15) = 33$ m ≈ 11 floors, but only 13 m of usable depth. At $d_f = 12$ m: $h_{\max} = 1.5 \times (12 + 15) = 40.5$ m ≈ 13 floors, but only 8 m of depth—barely enough for a single apartment depth plus corridor. The optimal d_f lies somewhere in this range, balancing height against usable floor area.*

3.3.2 Setbacks (Lateral and Rear)

Setbacks create mandatory buffer zones between the building and the lot boundary. The front setback is already captured by the facade setback variable d_f , so the remaining setback constraints apply to the *lateral* and *rear* boundaries of the lot. These serve multiple urban planning purposes: lateral separations (*distanciamiento*) ensure privacy between neighboring buildings, allow natural ventilation, and provide emergency access corridors; rear setbacks ensure minimum open space at the back of the lot.

A critical feature of Chilean setback regulations is that lateral separations *increase with building height*. A 3-story building might require only a 4 m lateral separation, while a 10-story building on the same lot might require 8 m. This height-dependent setback is what causes the buildable footprint to shrink at higher floors—the fundamental mechanism that produces non-rectangular building envelopes.

Let $\{\ell_k\}_{k \in K_{\text{lat}} \cup K_{\text{rear}}}$ denote the lateral and rear boundary segments (excluding the street-facing segments already handled by d_f). Define the Euclidean distance from a grid cell to a boundary segment:

$$\text{dist}(i, j; \ell_k) = \min_{p \in \ell_k} \|(i\delta, j\delta) - p\|_2.$$

The setback constraint is:

$$H_{\text{setback}}(i, j; d_f) = \begin{cases} 0 & \text{if } \exists k : \text{dist}(i, j; \ell_k) < d_k(z) \text{ for } z \in \mathbb{N} \\ \max\{z \in \mathbb{N} : \text{dist}(i, j; \ell_k) \geq d_k(z), \forall k \in K_{\text{lat}} \cup K_{\text{rear}}\} & \text{otherwise,} \end{cases} \quad (5)$$

where $d_k(z)$ is the setback distance at floor z . For lateral separations, this typically follows a linear formula: $d_k(z) = d_k^{(0)} + \alpha \cdot z \cdot h_f$, where $d_k^{(0)}$ is the base separation (often 4 m) and α is a slope parameter (often derived from the rasante angle). Note that the lateral setback constraint is independent of d_f (it depends on distance to lateral/rear boundaries, not to the street), but we write $H_{\text{setback}}(i, j; d_f)$ because the domain $G(d_f)$ changes with d_f .

3.3.3 Rasante (Inclined Plane)

The *rasante* is perhaps the most distinctive constraint in Chilean building regulation. It defines a fixed inclined plane that rises from each boundary segment at a prescribed angle from the horizontal, and the building volume must remain entirely below all such planes. Geometrically, the rasante creates a pyramidal or wedge-shaped envelope: the building can be tallest at the center of the lot and must taper down as it approaches any boundary.

The rasante is a *fixed geometric constraint*: the inclined plane is defined by the lot boundary position and the prescribed angle, and it does not depend on where the building facade is placed. However, it interacts with the facade setback indirectly: when the building is pushed further from the street, the front portion of the lot (between the street and the facade) is empty, so the rasante from the street-facing boundary is no longer binding at those cells. The rasante from lateral and rear boundaries, however, remains fully active.

The rasante angle is set by the zoning plan and typically ranges from 60° to 80° from horizontal. A steeper angle (closer to 90°) is more permissive, allowing taller buildings closer to the boundary; a shallower angle is more restrictive. In many Chilean zones, the rasante is measured from a point 2–3 m above the boundary (the “base height”), which allows a minimum building height even at the lot edge.

Let b_k be the base height at segment ℓ_k (typically the street level plus the rasante base offset, often 2–3 m). The maximum height at (i, j) due to the rasante from segment ℓ_k is:

$$H_{\text{rasante}}^{(k)}(i, j) = \left\lfloor \frac{b_k + \text{dist}(i, j; \ell_k) \cdot \tan(\theta)}{h_f} \right\rfloor. \quad (6)$$

The combined rasante constraint is the minimum over all boundary segments, since the building must lie below the rasante plane from *every* direction:

$$H_{\text{rasante}}(i, j; d_f) = \min_{k=1, \dots, K} H_{\text{rasante}}^{(k)}(i, j), \quad \forall (i, j) \in G(d_f). \quad (7)$$

Note that $H_{\text{rasante}}^{(k)}$ itself does not depend on d_f (the inclined plane is fixed in space), but its effect on the building changes because cells in $G(d_f)$ that are far from the street benefit from a higher rasante allowance at their position. When the building is set further back, the cells that are actually built upon are further from the street boundary, where the rasante plane is higher. This is a second mechanism (beyond the explicit height-distance regulation) by which increased facade setback allows taller construction.

Example 3 (Rasante Effect). *Consider a rectangular lot 30 m deep with rasante angle 70° and base height 3 m from the street boundary. With facade setback $d_f = 5$ m, the facade line is at 5 m from the boundary, where the rasante allows $3 + 5 \cdot \tan(70) = 16.7$ m ≈ 5 floors. With $d_f = 10$ m, the facade is at 10 m, where the rasante allows $3 + 10 \cdot 2.75 = 30.5$ m ≈ 10 floors. At the center of the lot (15 m from the boundary), the rasante allows $3 + 15 \cdot 2.75 = 44.2$ m ≈ 14 floors regardless of d_f , but this cell is only buildable if $d_f \leq 15$ m.*

3.3.4 Shadow Projection

Shadow regulations prevent buildings from casting excessive shadows onto public spaces during certain hours and seasons. These constraints are particularly relevant in the southern hemisphere,

where the sun tracks across the northern sky: a tall building on the north side of an east-west street can cast deep shadows onto the street and neighboring lots to the south.

The shadow constraint is evaluated at specific solar positions (typically at solar noon and critical hours during the winter solstice and equinoxes, as prescribed by the local zoning ordinance). For each regulated solar position, the shadow cone of a vertical building element at height h is computed by projecting along the sun vector.

Let $\mathcal{S}_d$ denote the shadow cone operator for solar direction $d \in \{\text{summer, equinox, winter}\}$. For a building element of height h at position (x, y) , the shadow cone projects onto the ground plane as:

$$\text{Shadow}_d(x, y, h) = \{(x', y') \in \mathbb{R}^2 : (x', y') = (x, y) + h \cdot \mathbf{v}_d(\tau), \tau \in [0, 1]\}, \quad (8)$$

where $\mathbf{v}_d(\tau)$ is the shadow direction vector at time τ for season d , computed from the solar azimuth and elevation angle at the lot's latitude. Let Ω_{pub} be the set of public spaces (streets, plazas, parks) whose regulated zones must remain shadow-free during prescribed hours. The shadow constraint limits the height at each cell to the tallest value whose shadow does not violate the regulation:

$$H_{\text{shadow}}(i, j; d_f) = \max\{z \in \mathbb{N} : \text{Shadow}_d(i\delta, j\delta, z \cdot h_f) \cap \Omega_{\text{pub}} = \emptyset, \forall d\}. \quad (9)$$

The shadow constraint also interacts with the facade setback: when the building is set further back, its shadow-casting facade is further from the street, so the shadow must travel a longer distance before reaching the regulated public space. This allows taller construction at the facade line. Conversely, for cells deep inside the lot (far from the street), the shadow constraint is typically non-binding because their shadows fall onto the lot itself.

3.4 Optimal Facade Setback Selection

The central optimization in Stage A is the selection of the facade setback distance d_f^* that maximizes the total building value. For each candidate setback, the parametric height map $H(\cdot; d_f)$ yields a set of floor footprints, which in turn determine the floor plan values (computed in Stages B and C). The optimization is:

$$d_f^* = \arg \max_{d_f \in \{d_f^{\min}, d_f^{\min} + \delta, \dots, d_f^{\max}\}} \text{BuildingValue}(H(\cdot; d_f)), \quad (10)$$

where $d_f^{\max}$ is a practical upper bound on the setback (beyond which the remaining footprint is too small for a viable building), and $\text{BuildingValue}(\cdot)$ is the total profit from Stages B and C evaluated on the height map.

The search space is small: with $\delta = 0.5$ m resolution and a typical range of 3–15 m, there are at most 25 candidate values. For each candidate, the height map computation is $O(|G| \cdot |\mathcal{R}| \cdot K)$, and the subsequent floor plan optimization (Stages B and C) is sub-millisecond. The entire facade optimization therefore completes in well under one second.

Remark 2 (Unimodality). *In practice, the total building value as a function of d_f is typically unimodal: it increases as additional floors more than compensate for lost footprint area, reaches a peak, and then decreases as the footprint becomes too narrow for efficient apartment packing. This unimodality allows the use of golden-section search instead of exhaustive enumeration, reducing the number of evaluations to $O(\log(1/\delta))$. However, given the small search space, exhaustive evaluation at every δ -step is equally practical and avoids assumptions about the value function shape.*

3.5 Floor Footprint Extraction

Once the optimal facade setback d_f^* is selected, the height map $H(\cdot) \equiv H(\cdot; d_f^*)$ is fixed, and the floor footprints are extracted. The footprint at floor level z consists of all buildable cells where the height map meets or exceeds z :

$$\mathcal{F}_z = \{(i, j) \in G(d_f^*) : H(i, j; d_f^*) \geq z\}, \quad z = 1, \dots, Z_{\max}, \quad (11)$$

where $Z_{\max} = \max_{(i,j) \in G(d_f^*)} H(i, j; d_f^*)$ is the maximum building height anywhere on the lot under the chosen setback.

By construction, the footprints form a nested (monotone decreasing) sequence:

$$\mathcal{F}_1 \supseteq \mathcal{F}_2 \supseteq \dots \supseteq \mathcal{F}_{Z_{\max}}.$$

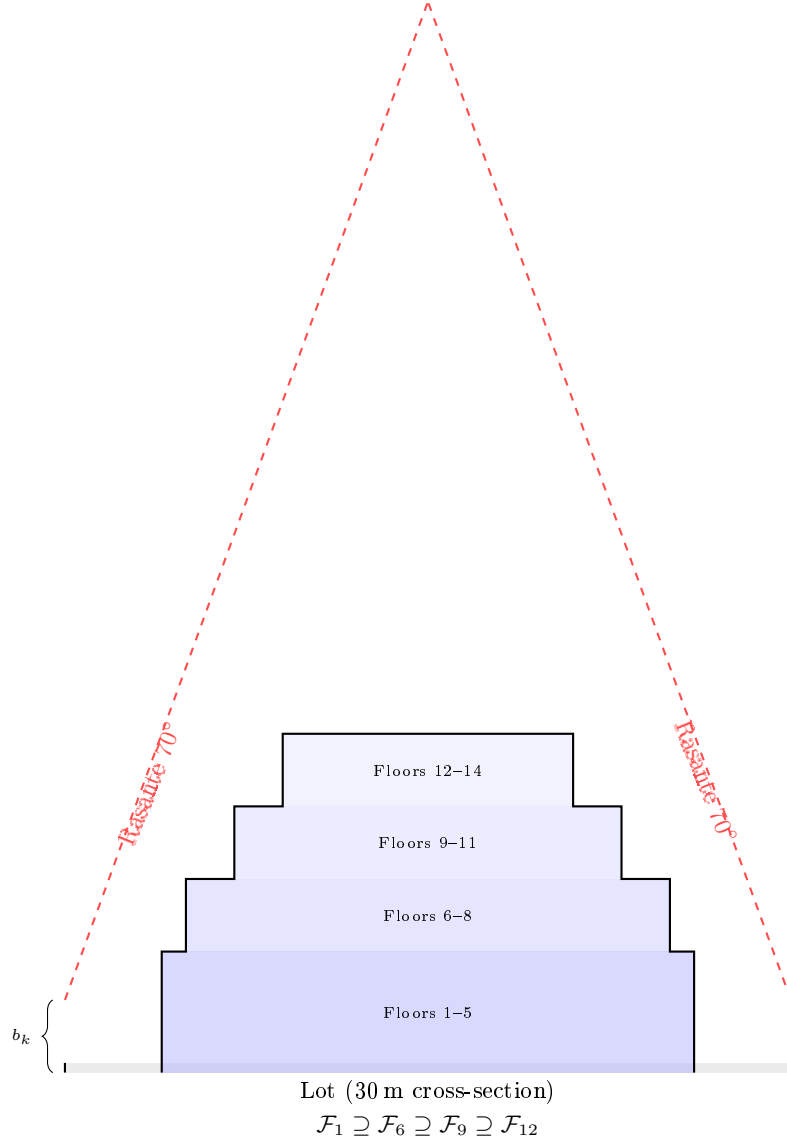


Figure 3: Building cross-section shaped by the rasante constraint (not to scale). The rasante planes from both lateral boundaries create a tapered envelope. The building steps back at higher floors, producing nested footprints $\mathcal{F}_1 \supseteq \mathcal{F}_6 \supseteq \mathcal{F}_9 \supseteq \mathcal{F}_{12}$. Each distinct footprint defines a group of floors with identical plans.

Lower floors have larger footprints because the lateral setback and rasante constraints are less restrictive close to ground level; upper floors have progressively smaller footprints as the inclined

planes and height-dependent separations take effect. The front boundary of every footprint is at distance d_f^* from the street (the facade line), while the lateral and rear boundaries recede inward at higher floors.

We define the set of *distinct footprints*—footprints that define genuinely different polygons—since consecutive floors often share the same footprint (e.g., floors 3 through 7 might all have identical outlines because no regulation changes in that height range):

$$\mathcal{D} = \{\mathcal{F}_z : z \in \{1, \dots, Z_{\max}\}\} / \sim, \quad (12)$$

where $\mathcal{F}_z \sim \mathcal{F}_{z'}$ if they define the same polygon. Let $|\mathcal{D}|$ denote the number of distinct footprints. For each $\mathcal{F} \in \mathcal{D}$, let $n(\mathcal{F})$ be the number of floors sharing that footprint.

In practice, $|\mathcal{D}|$ is small—typically 2 to 5 for buildings up to 15 floors. This is because the regulations change in discrete steps (a floor either fits within the *rasante* or it doesn't), so the footprint changes only at a few critical heights.

3.6 Ground Occupation Constraint

The ground occupation coefficient limits how much of the lot can be covered by the building at ground level. This constraint ensures permeable surfaces, green areas, and pedestrian circulation at grade.

Let c_{occ} be the maximum ground occupation coefficient. The ground floor footprint must satisfy:

$$\text{Area}(\mathcal{F}_1) \leq c_{\text{occ}} \cdot \text{Area}(P). \quad (13)$$

If this constraint is violated (which can occur when the facade setback d_f^* produces a large ground-level footprint), cells in $\mathcal{F}_1$ are removed starting from the interior cells furthest from the corridor axis (preserving connectivity and compactness) until the constraint is satisfied. This may also constrain the facade setback search: candidates d_f for which even the reduced $\mathcal{F}_1$ is architecturally impractical are discarded.

4 Stage B: Floor Plan Optimization

With the buildable envelope computed, the next stage addresses the core combinatorial challenge: determining the optimal internal layout of each floor. In a corridor-based residential building—the dominant typology for mid-rise urban development—each floor consists of a central corridor providing access to apartments arranged on both sides, a circulation core (staircase and elevator shaft), and possibly terraces or common areas at the ends.

The key geometric observation is that apartments in these buildings are arranged in a *linear sequence* along the corridor. Each apartment occupies a rectangular slot defined by its *frontage width* (parallel to the corridor) and *depth* (perpendicular to the corridor, extending to the facade). This linear arrangement means that once the corridor position and orientation are fixed, the problem of assigning apartments to each side of the corridor reduces to a *one-dimensional packing problem*—specifically, a bounded knapsack.

However, real floor footprints are not always rectangular. The height-dependent setbacks from Stage A commonly produce L-shaped, U-shaped, H-shaped, or T-shaped footprints, especially at lower floors of corner lots or lots with irregular boundaries. A corridor in an L-shaped floor must turn a corner; an H-shaped floor may require a branching corridor. The medial axis of the floor polygon provides a natural and mathematically principled way to determine where the corridor should run.

For each distinct footprint $\mathcal{F} \in \mathcal{D}$, we optimize the floor plan independently. The optimization proceeds in four sub-stages.

4.1 Stage B.1: Skeleton Graph and Corridor Routing

4.1.1 Rectilinear Medial Axis

The first step is to determine where the corridor should run within the floor footprint. Intuitively, the corridor should pass through the “center” of the polygon so that apartments on both sides have approximately equal and maximal depth. The mathematical tool for this is the *medial axis*—the set of points inside the polygon that are equidistant from at least two boundary edges. For a rectangular footprint, the medial axis is simply the centerline; for an L-shaped footprint, it is an L-shaped line running through the center of each arm.

Let $\mathcal{F}$ be a rectilinear polygon (which it will be, since the grid discretization produces axis-aligned boundaries). Its *medial axis* $M(\mathcal{F})$ is the locus of points equidistant from the two nearest boundary edges. For rectilinear polygons, $M(\mathcal{F})$ is a rectilinear tree—a tree graph where all segments are axis-aligned (horizontal or vertical).

Definition 3 (Skeleton Graph). *The skeleton graph $\mathcal{G} = (V, E)$ is a graph derived from the medial axis by snapping its vertices to a structural grid:*

- V is the set of vertices of $M(\mathcal{F})$ snapped to the structural grid $\Gamma = \{(x, y) : x \in \Gamma_x, y \in \Gamma_y\}$, where Γ_x and Γ_y are the column and beam positions (typically spaced 6–8 m apart, reflecting standard reinforced-concrete structural spans),
- E is the set of axis-aligned segments connecting adjacent vertices along $M(\mathcal{F})$.

The snapping to a structural grid is architecturally motivated: in reinforced-concrete construction (the dominant method for mid-rise residential buildings), columns must be arranged on a regular grid for efficient load transfer. The corridor axis naturally aligns with one direction of this grid, and apartment partition walls align with the other.

In practice, $|V| \leq 6$ and $|E| \leq 8$ for the floor shapes encountered in urban development (L, H, U, T, and rectangular). This small size is a consequence of the fact that urban lots, while irregular, produce floor footprints that are compositions of a small number of rectangular blocks.

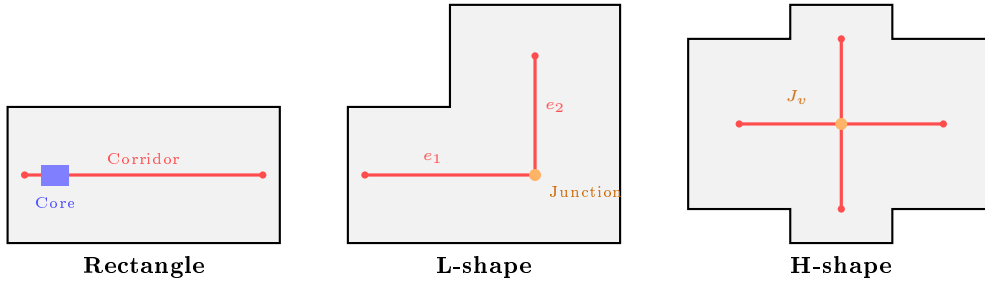


Figure 4: Skeleton graphs (red) for three common floor plan shapes. Vertices (dots) mark corridor endpoints and junctions; edges define corridor segments. The **rectangle** has a single straight corridor. The **L-shape** has a two-edge corridor with one junction (orange) at the corner. The **H-shape** has a cross-shaped corridor with a central junction. The blue square marks a possible circulation core placement.

4.1.2 Corridor as Connected Subtree

A *corridor* is a connected subtree $\mathcal{C} \subseteq \mathcal{G}$ with uniform width w_c (typically 1.5–2.0 m, set by building code requirements for emergency egress). The corridor polygon is obtained by buffering the subtree by half the corridor width:

$$P_{\mathcal{C}} = \{p \in \mathbb{R}^2 : \text{dist}(p, \mathcal{C}) \leq w_c/2\}.$$

Not every connected subtree of $\mathcal{G}$ constitutes a usable corridor. A valid corridor must reach all parts of the floor footprint (so that every apartment has access) and must accommodate the circulation core (staircase and elevator) at one of its vertices.

Definition 4 (Valid Corridor). *A corridor $\mathcal{C}$ is valid if:*

- (i) **Coverage:** *Every point in the floor footprint is within a maximum access distance d_{acc} from the corridor:*

$$\max_{p \in \mathcal{F}} \text{dist}(p, P_{\mathcal{C}}) \leq d_{acc}. \quad (14)$$

This ensures that no apartment is located unreasonably far from the corridor, as required by fire safety codes and practical livability.

- (ii) **Circulation core placement:** *$\mathcal{C}$ must contain at least one vertex $v \in V$ where a circulation core of dimensions $w_{core} \times d_{core}$ (typically $6 \text{ m} \times 5 \text{ m}$, accommodating one elevator and one fire staircase) can be placed without overlapping the lot boundary.*

Let $\mathcal{C}^* = \{\mathcal{C}_1, \dots, \mathcal{C}_M\}$ be the set of all valid corridors. Since $\mathcal{G}$ is small, the enumeration of all connected subtrees is computationally trivial: $|\mathcal{C}^*|$ is typically 2–10.

Example 4 (Corridor Topologies). *For a **rectangular** footprint, there is typically one corridor option: a single straight segment running the length of the rectangle. For an **L-shaped** footprint, there are typically 2–3 options: (a) an L-shaped corridor following both arms, (b) a corridor running along only the longer arm, and (c) a corridor running along only the shorter arm. Option (a) serves the entire floor but introduces a junction (corner); options (b) and (c) leave part of the floor unserved but avoid the junction. For an **H-shaped** footprint, the corridor typically has a T or cross configuration, with junctions at each branching point.*

4.2 Stage B.2: Strip Decomposition

Once a corridor is chosen, the usable floor area naturally divides into rectangular *strips*—the bands of space on each side of each corridor segment where apartments can be placed. This decomposition is the crucial step that reduces the two-dimensional floor plan layout to a collection of independent one-dimensional packing problems.

4.2.1 Strip Segments

Given a valid corridor $\mathcal{C}_m \in \mathcal{C}^*$, the usable floor area (everything that is not corridor, circulation core, or structural elements) is:

$$U_m = \mathcal{F} \setminus P_{\mathcal{C}_m}.$$

Each edge $e \in E(\mathcal{C}_m)$ is an axis-aligned segment. For a horizontal edge e at ordinate y_e spanning the interval $[x_e^L, x_e^R]$, the corridor creates two strips—one on the north side and one on the south side:

$$S_e^+ = U_m \cap \{(x, y) : x \in [x_e^L, x_e^R], y \in [y_e + w_c/2, y_e + w_c/2 + D_{\max}]\}, \quad (15)$$

$$S_e^- = U_m \cap \{(x, y) : x \in [x_e^L, x_e^R], y \in [y_e - w_c/2 - D_{\max}, y_e - w_c/2]\}, \quad (16)$$

where $D_{\max}$ is the maximum apartment depth (interior depth plus terrace, typically 12–16 m, bounded by natural light penetration requirements). Vertical edges produce analogous east/west strips.

The intersection with U_m in the definitions above is important: it clips the strips to the actual footprint boundary, so that a strip along a corridor edge near the lot boundary may be shallower than $D_{\max}$.

Each strip S is characterized by three key parameters:

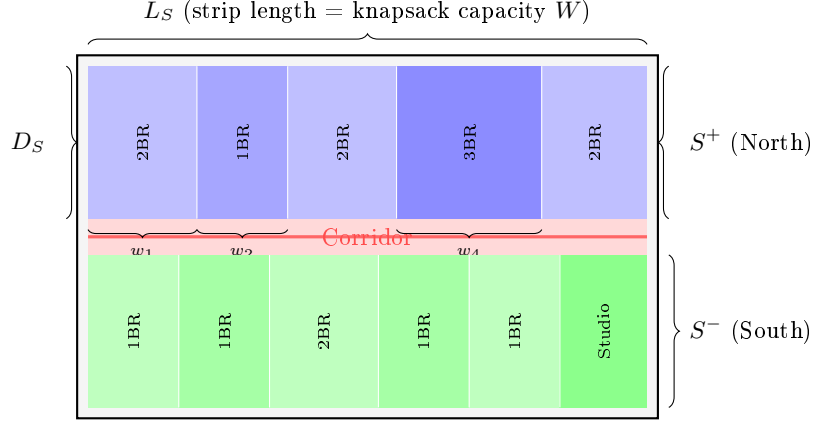


Figure 5: Strip decomposition of a rectangular floor plan. The corridor (red) divides the floor into two strips: S^+ (north-facing) and S^- (south-facing). Each strip is packed independently with apartment templates via the bounded knapsack: template widths w_t consume strip length L_S , while template depths must fit within strip depth D_S . Different shading intensities indicate different apartment types.

- **Length L_S :** the dimension parallel to the corridor edge. This determines how many apartments can fit side by side—it is the “capacity” of the knapsack.
- **Depth D_S :** the dimension perpendicular to the corridor (available for apartment depth + terrace). This determines which apartment templates are feasible—only templates whose total depth (interior + terrace) fits within D_S can be placed.
- **Orientation $\theta_S \in \{\text{N, S, E, W, NE, NW, SE, SW}\}$:** the outward-facing direction of the strip, determined by which side of the corridor the strip lies on and the compass bearing of the lot boundary. Orientation affects apartment revenue through market preferences (north-facing apartments command premium prices in the southern hemisphere due to superior solar exposure).

Strips with $L_S < w_{\min}^{\text{apt}}$ (minimum apartment width, typically 5–6 m) or $D_S < d_{\min}^{\text{apt}}$ (minimum apartment depth, typically 4–5 m) are discarded as too small to accommodate any apartment template.

4.2.2 Junction Zones

Where the corridor changes direction—at each vertex of the corridor subtree with degree ≥ 2 —a *junction zone* arises. These are the irregular regions at corridor intersections where the simple strip model does not directly apply. The design treatment of each junction significantly affects the overall floor plan quality and must be handled explicitly.

A *junction* occurs at each vertex $v \in V(\mathcal{C}_m)$ with degree ≥ 2 . The junction zone J_v is the rectangular region at the corridor intersection:

$$J_v = \{(x, y) : |x - x_v| \leq r_J, |y - y_v| \leq r_J\} \cap \mathcal{F}, \quad (17)$$

where r_J is the junction radius (typically half the structural grid spacing, i.e., 3–4 m). Let $\mathcal{J}_m = \{J_1, \dots, J_{|\mathcal{J}_m|}\}$ be the set of junctions for corridor $\mathcal{C}_m$.

Each junction J_v admits a discrete set of *junction options* $\mathcal{O}_v$, representing the different architectural treatments of the junction zone:

- **Circulation core:** Place the staircase and elevator shaft at the junction. This is often the architecturally natural location for the core, as it provides equal access to all corridor branches. However, it consumes area from adjacent strips (reducing their usable length) and generates no direct revenue. Revenue contribution: 0.

- (b) **Corner apartment:** An L-shaped apartment that wraps around the corridor turn, with frontage on two perpendicular corridor edges. These units are architecturally desirable because they have dual exposure (windows facing two directions), commanding a market premium. However, they require sufficient depth in both adjacent strips and introduce a non-rectangular floor plan for that unit. Revenue contribution: r_{corner} (premium for dual exposure, typically 5–15% above a standard unit of the same area).
- (c) **Common area:** Leave the junction as lobby space, a small lounge, or an amenity area (mailroom, bicycle parking access). This does not consume strip length or generate revenue, but may be required for building code compliance (e.g., a minimum lobby area per floor) or market positioning. Revenue contribution: 0, but no strip area consumed either.

Each option $o \in \mathcal{O}_v$ reduces the available length of adjacent strips by a known amount $\Delta L_{v,s}^{(o)}$ for each adjacent strip s . For instance, placing a circulation core of width 6 m at a junction reduces the length of each adjacent strip by 3 m (half the core, as the other half extends into the opposite strip).

4.3 Stage B.3: Apartment Template Packing

With the floor decomposed into strips of known length, depth, and orientation, the problem of placing apartments on each strip becomes a classical one-dimensional bounded knapsack problem. This is the computational heart of the floor plan optimization: it determines which apartments go where, and directly produces the economic value of each floor.

4.3.1 Apartment Templates

Unlike the current approach of generating apartments as generic rectangles and assigning types *after* optimization, our formulation defines apartment templates *from* market-driven type specifications. Each template represents a specific architectural configuration with known dimensions, area, and market value.

Definition 5 (Apartment Template). *An apartment template is a tuple $t = (\tau, w_t, d_t, d_t^{\text{ter}}, a_t, a_t^{\text{ter}}, r_t)$ where:*

- $\tau \in \mathcal{T}$ is the apartment type (e.g., studio, 1BR, 2BR, 3BR), each with specific architectural requirements (number of bedrooms, bathrooms, minimum kitchen area),
- w_t is the frontage width (parallel to corridor), constrained by bedroom layout: a studio may need only 5 m of frontage, while a 3BR apartment typically needs 9–12 m to accommodate three bedrooms side by side,
- d_t is the interior depth (perpendicular to corridor), bounded above by natural light requirements: a depth of 12–14 m is the practical maximum for single-exposure residential units, as interior spaces beyond this distance receive insufficient daylight,
- d_t^{ter} is the terrace depth,
- $a_t = w_t \cdot d_t$ is the interior area,
- $a_t^{\text{ter}} = w_t \cdot d_t^{\text{ter}}$ is the terrace area,
- r_t is the total revenue, computed from market prices per usable square meter.

The set of all templates is denoted $\mathcal{T}_{\text{all}}$. For a strip S with depth D_S , the *feasible templates* are those whose total depth (interior + terrace) fits within the available strip depth:

$$\mathcal{T}_S = \{t \in \mathcal{T}_{\text{all}} : d_t + d_t^{\text{ter}} \leq D_S\}. \quad (18)$$

Templates are generated from market data: for each apartment type $\tau \in \mathcal{T}$, the width and depth ranges are bounded by architectural constraints that reflect real design practice:

$$w_{\tau}^{\min} \leq w_t \leq w_{\tau}^{\max}, \quad d_{\tau}^{\min} \leq d_t \leq d_{\tau}^{\max}, \quad a_{\tau}^{\min} \leq a_t \leq a_{\tau}^{\max}. \quad (19)$$

For example, a 2BR apartment might have $w \in [7, 10]$ m, $d \in [7, 12]$ m, and $a \in [50, 75]$ m². The discrete template set is generated by sampling these ranges at a resolution of $\delta_w = 0.5$ m and filtering to keep only architecturally valid configurations (e.g., aspect ratios between 0.5 and 2.0, area within type-specific bounds).

The terrace depth is determined by interpolation from market-typical terrace areas:

$$d_t^{\text{ter}} = \text{clamp}\left(\frac{A_{\text{ter}}(a_t)}{w_t}, d_{\text{ter}}^{\min}, d_{\text{ter}}^{\max}\right), \quad (20)$$

where $A_{\text{ter}}(\cdot)$ is a piecewise-linear interpolation of typical terrace areas by interior area, derived from market transaction data. Smaller apartments tend to have proportionally smaller terraces; larger units may have terraces of 10–20 m².

The revenue per template incorporates an orientation quality factor that reflects market preferences for solar exposure:

$$r_t(\theta_S) = (a_t + 0.5 a_t^{\text{ter}}) \cdot p_\tau \cdot \phi(\theta_S), \quad (21)$$

where p_τ is the price per usable square meter for type τ (derived from comparable market transactions), and $\phi(\theta_S) \in (0, 1]$ is an orientation quality factor. In the Chilean market, the factor 0.5 on terrace area reflects the convention that terraces contribute half their area to the “usable surface” for pricing purposes (matching the legal definition in Chilean real estate law). Typical orientation factors are $\phi(\text{N}) = 1.0$ (full northern sun), $\phi(\text{NE}) = \phi(\text{NW}) = 0.95$, $\phi(\text{E}) = \phi(\text{W}) = 0.90$, $\phi(\text{S}) = 0.85$ (least desirable in the southern hemisphere).

4.3.2 Bounded Knapsack Formulation

For a single strip S with length L_S (discretized to $W = \lfloor L_S/\delta_w \rfloor$ units, where $\delta_w = 0.5$ m is the width discretization step), the apartment packing is a *bounded knapsack problem*: we seek to fill a knapsack of capacity W with items (apartment templates), where each item has a known size (discretized width) and value (revenue), and each item type has an upper bound on the number of copies.

Sets and indices.

- $\mathcal{T}_S$: set of feasible templates for strip S , indexed by t ,
- W : strip capacity (discretized length in units of δ_w).

Parameters.

- $w'_t = \lceil w_t/\delta_w \rceil$: discretized width of template t (in units of δ_w),
- r_t : revenue of template t (from Equation (21)),
- u_t : upper bound on copies of template t in strip S , given by $u_t = \lfloor W/w'_t \rfloor$ (at most as many copies as would fill the entire strip).

Decision variables.

$$n_t \in \{0, 1, \dots, u_t\}, \quad \forall t \in \mathcal{T}_S.$$

Here n_t represents the number of copies of template t placed in the strip.

Formulation.

$$\max \quad \sum_{t \in \mathcal{T}_S} r_t \cdot n_t \quad (22a)$$

$$\text{s.t.} \quad \sum_{t \in \mathcal{T}_S} w'_t \cdot n_t \leq W, \quad (22b)$$

$$n_t \in \{0, 1, \dots, u_t\}, \quad \forall t \in \mathcal{T}_S. \quad (22c)$$

The objective (22a) maximizes total revenue from the apartments placed in the strip. Constraint (22b) ensures that the total width of placed apartments does not exceed the strip length. Constraint (22c) bounds the number of copies of each template type.

This is the classical bounded knapsack problem, solvable exactly by dynamic programming in $O(|\mathcal{T}_S| \cdot W)$ time and $O(W)$ space. The Bellman recursion is:

$$V(w) = \max_{t \in \mathcal{T}_S} \{V(w - w'_t) + r_t : w'_t \leq w, n_t < u_t\}, \quad V(0) = 0. \quad (23)$$

The optimal strip value is $V(W)$, and the optimal apartment assignment is recovered by standard backtracking through the DP table. For typical parameter values ($|\mathcal{T}_S| \leq 60$ templates, $W \leq 240$ half-meter units for a 120 m-long strip, which is an extreme case), the DP solves in microseconds.

4.3.3 Market Mix via Lagrangian Relaxation

The per-strip knapsack formulation maximizes revenue independently for each strip, without regard for the overall building mix. In practice, the market imposes soft constraints on the apartment type distribution: a building with 80% studios and 20% 3BR apartments may be optimal strip-by-strip but unsellable as a whole product. The developer needs a balanced mix that appeals to the target market segment.

Let $q_\tau^{\min}$ and $q_\tau^{\max}$ be the minimum and maximum fraction of units of type τ in the building (e.g., “at least 20% and at most 40% of units should be 2BR”). Define the building-wide mix constraint:

$$q_\tau^{\min} \cdot N_{\text{total}} \leq N_\tau \leq q_\tau^{\max} \cdot N_{\text{total}}, \quad \forall \tau \in \mathcal{T}, \quad (24)$$

where $N_\tau = \sum_S \sum_{t: \tau(t)=\tau} n_t^{(S)} \cdot n_f(S)$ is the total number of units of type τ across all strips and floors (with $n_f(S)$ being the number of floors that share the footprint containing strip S), and $N_{\text{total}} = \sum_\tau N_\tau$.

These constraints couple all strips and all floors, breaking the separability of the knapsack decomposition. We restore separability via Lagrangian relaxation. Introduce dual multipliers $\lambda_\tau^+, \lambda_\tau^- \geq 0$ for the upper and lower mix bounds, and modify the per-template revenue in each strip’s knapsack:

$$\tilde{r}_t = r_t - \lambda_{\tau(t)}^+ + \lambda_{\tau(t)}^-. \quad (25)$$

Intuitively, if the current solution has too many studios ($N_{\text{studio}} > q_{\text{studio}}^{\max} \cdot N_{\text{total}}$), the multiplier $\lambda_{\text{studio}}^+$ increases, penalizing studio templates and encouraging the strip knapsacks to select other types instead. Conversely, if there are too few 3BR units, λ_{3BR}^- increases, effectively subsidizing 3BR templates to make them more attractive relative to smaller types.

The Lagrangian subproblem decomposes by strip (each strip solves (22) with revenues $\tilde{r}_t$). The multipliers are updated by subgradient ascent:

$$\lambda_\tau^+ \leftarrow \max(0, \lambda_\tau^+ + \alpha_k (N_\tau - q_\tau^{\max} \cdot N_{\text{total}})), \quad (26)$$

$$\lambda_\tau^- \leftarrow \max(0, \lambda_\tau^- + \alpha_k (q_\tau^{\min} \cdot N_{\text{total}} - N_\tau)), \quad (27)$$

where α_k is a diminishing step size (e.g., $\alpha_k = \alpha_0/\sqrt{k}$). Convergence to a near-optimal feasible solution is typically achieved in 10–20 iterations, each iteration requiring only the re-resolution of all strip knapsacks (microseconds each).

4.4 Stage B.4: Joint Floor Optimization

The previous sub-stages—corridor routing, strip decomposition, junction treatment, and apartment packing—must be combined into a single optimal floor plan. Since the number of corridor topologies is small and the number of junction options per junction is at most 3, we solve this by exhaustive enumeration: for every valid corridor and every combination of junction treatments, we solve all strip knapsacks and compute the total floor value, then select the best combination.

Sets.

- $\mathcal{C}^*$: set of valid corridors for $\mathcal{F}$,
- $\mathcal{S}_m$: set of strips induced by corridor $\mathcal{C}_m$,
- $\mathcal{J}_m$: set of junctions for corridor $\mathcal{C}_m$,
- $\mathcal{O}_v$: set of junction options for junction J_v .

Joint optimization. For each corridor $\mathcal{C}_m \in \mathcal{C}^*$ and each combination of junction options $\mathbf{o} = (o_v)_{v \in \mathcal{J}_m} \in \prod_{v \in \mathcal{J}_m} \mathcal{O}_v$, the junction choices modify the available strip lengths. The adjusted strip length accounts for the area consumed by each junction treatment:

$$L_S^{(\mathbf{o})} = L_S - \sum_{v \in \mathcal{J}_m} \Delta L_{v,S}^{(o_v)}, \quad \forall S \in \mathcal{S}_m. \quad (28)$$

The total floor value for corridor $\mathcal{C}_m$ and junction options $\mathbf{o}$ is the sum of junction revenues and strip packing values:

$$\Phi(\mathcal{C}_m, \mathbf{o}) = \sum_{v \in \mathcal{J}_m} r^{(o_v)} + \sum_{S \in \mathcal{S}_m} V_S(L_S^{(\mathbf{o})}), \quad (29)$$

where $r^{(o_v)}$ is the revenue from junction option o_v (positive for corner apartments, zero for circulation cores and common areas) and $V_S(\cdot)$ is the knapsack value function from (23) evaluated at the adjusted length.

The optimal floor plan is obtained by selecting the corridor and junction combination that maximizes the total floor value:

$$(\mathcal{C}^*, \mathbf{o}^*) = \arg \max_{\mathcal{C}_m \in \mathcal{C}^*, \mathbf{o} \in \prod_v \mathcal{O}_v} \Phi(\mathcal{C}_m, \mathbf{o}). \quad (30)$$

The total number of combinations to evaluate is $|\mathcal{C}^*| \cdot \prod_v |\mathcal{O}_v|$. Table 1 shows that this remains very small for all common floor shapes:

Floor shape	$ \mathcal{C}^* $	Junctions	$ \mathcal{O}_v $	Total combinations
Rectangle	1–2	0	—	1–2
L-shape	2–3	1	3	6–9
T-shape	2–4	1–2	3	6–36
U-shape	2–4	2	3	18–36
H-shape	3–5	2	3	27–45

Table 1: Search space size for the joint floor optimization by floor shape. Even the most complex shape (H) requires evaluating at most 45 combinations, each consisting of a handful of microsecond knapsack solves.

Each combination requires solving $|\mathcal{S}_m|$ knapsack instances (each in microseconds via the DP of (23)), making the full enumeration computationally trivial—the entire joint optimization for one floor plan completes in well under a millisecond.

5 Stage C: Multi-floor Assembly

The final stage assembles the optimized floor plans into a complete building and enforces the aggregate regulatory constraints that span the entire structure. While Stages A and B operate at the level of individual grid cells and individual floors, respectively, Stage C addresses building-level constraints: the total constructibility (floor area ratio), the maximum density (dwelling units per hectare), and the parking requirements.

5.1 Floor Grouping

As established in Section 3, floors sharing the same footprint use the same floor plan. This is both a computational simplification (we solve Stage B once per distinct footprint, not once per floor) and a structural requirement (maintaining a consistent column grid across floors is essential for reinforced-concrete construction).

Let $\mathcal{D} = \{\mathcal{F}^{(1)}, \dots, \mathcal{F}^{(D)}\}$ be the distinct footprints, with n_d floors using footprint $\mathcal{F}^{(d)}$. The floor plan for $\mathcal{F}^{(d)}$ is denoted $\Pi^{(d)}$, and its per-floor metrics are:

- $a_{\text{int}}^{(d)}$: total interior apartment area per floor,
- $a_{\text{ter}}^{(d)}$: total terrace area per floor,
- $N_{\tau}^{(d)}$: number of units of type τ per floor,
- $a_{\text{util}}^{(d)} = a_{\text{int}}^{(d)} + 0.5 a_{\text{ter}}^{(d)}$: total usable area per floor (following the Chilean convention that terraces count at 50% for constructibility calculations).

5.2 Building-wide Constraints

Constructibility. The constructibility coefficient c_{con} limits the total built area (summed across all floors) relative to the lot area. This is the single most important regulatory constraint on building size, as it directly caps the total sellable area:

$$\sum_{d=1}^D n_d \cdot a_{\text{util}}^{(d)} \leq c_{\text{con}} \cdot A_P, \quad (31)$$

where $A_P = \text{Area}(P)$ is the lot area.

If this constraint is violated after assembling all floor plans (which can happen because Stage B maximizes per-floor value without considering the aggregate limit), the number of floors is reduced starting from the highest level—the smallest footprint, which typically has the lowest per-floor value—until the constraint is satisfied.

Density. The density constraint limits the total number of dwelling units, controlling population density in the neighborhood:

$$\sum_{d=1}^D n_d \cdot \sum_{\tau \in \mathcal{T}} N_{\tau}^{(d)} \leq \frac{\rho_{\text{max}}}{10\,000} \cdot A_B, \quad (32)$$

where ρ_{max} is the maximum density in dwelling units per hectare and A_B is the applicable density area (gross or net lot area, depending on the regulatory framework). The factor 10 000 converts from per-hectare to per-m² units.

Total units. The total number of dwelling units in the building is:

$$N_{\text{total}} = \sum_{d=1}^D n_d \cdot \sum_{\tau \in \mathcal{T}} N_{\tau}^{(d)}. \quad (33)$$

5.3 Parking Requirements

Parking is one of the largest cost drivers in urban residential development. Underground parking construction costs 3–5 times more per square meter than above-ground residential floors, and each underground level requires deep excavation, retaining walls, waterproofing, and mechanical ventilation. Accurate parking dimensioning is therefore critical to project feasibility.

Parking demand is derived from the apartment mix and applicable regulations. Let π_{τ} be the parking ratio for apartment type τ —the number of parking spaces required per apartment of

that type. In Chilean regulations, this is a function of apartment area: apartments below 50 m² may require fewer spaces than those above 140 m². The total parking requirement is:

$$N_{\text{park}} = \sum_{\tau \in \mathcal{T}} \pi_{\tau} \cdot N_{\text{total}, \tau} + N_{\text{visit}} + N_{\text{disabled}} - N_{\text{disc}}, \quad (34)$$

where N_{visit} is the number of visitor parking spaces (typically 1 per 10–20 apartments), N_{disabled} is the number of accessible parking spaces (required by disability access legislation), and N_{disc} is a discount applicable when the building is near a metro station or provides bicycle parking (as incentivized by some municipal plans).

5.3.1 Underground Parking Optimization

The underground parking area is constrained by the lot boundary with its own setback rules (underground setbacks are typically smaller than above-ground setbacks, as they do not affect light or privacy). Let $P_{\text{sub}} \subseteq P$ be the underground buildable polygon (lot polygon minus underground setbacks), and $A_{\text{sub}} = \text{Area}(P_{\text{sub}})$.

The required total parking area accounts for car spaces, bicycle parking, and storage units (one per apartment, a common market expectation in Chile):

$$A_{\text{park}} = N_{\text{park}} \cdot a_{\text{space}} + N_{\text{bike}} \cdot a_{\text{bike}} + N_{\text{storage}} \cdot a_{\text{storage}}, \quad (35)$$

where $a_{\text{space}} \approx 25\text{--}30$ m² per parking space (including circulation lanes and ramps), $a_{\text{bike}} \approx 1.5$ m² per bicycle spot, and $a_{\text{storage}} \approx 4\text{--}6$ m² per storage unit.

The number of underground levels is:

$$Z_{\text{sub}} = \left\lceil \frac{A_{\text{park}}}{\min(A_{\text{sub}}, c_{\text{occ}}^{\text{sub}} \cdot A_P)} \right\rceil, \quad (36)$$

where $c_{\text{occ}}^{\text{sub}}$ is the underground occupation coefficient (often higher than the above-ground coefficient, sometimes up to 100% for lots in dense zones).

If $Z_{\text{sub}} > 1$, the last underground level may have a reduced footprint (to avoid excavating a full level when only a fraction of the area is needed). Let $A_{\text{last}} = A_{\text{park}} - (Z_{\text{sub}} - 1) \cdot A_{\text{sub}}$. The last-level polygon is obtained by offsetting P_{sub} inward until its area equals A_{last} . This inward offset is computed by binary search on the offset distance ϵ : at each step, the polygon offset($P_{\text{sub}}, -\epsilon$) is computed and its area compared to A_{last} , converging in $O(\log(1/\text{tol}))$ iterations.

5.4 Total Building Value

The total building revenue is the sum of per-floor revenues across all floor groups:

$$R_{\text{total}} = \sum_{d=1}^D n_d \cdot \Phi^{(d)*}, \quad (37)$$

where $\Phi^{(d)*}$ is the optimal floor value from (30) for footprint $\mathcal{F}^{(d)}$.

The total development cost includes land acquisition, above-ground construction, underground construction, and project overhead:

$$C_{\text{total}} = C_{\text{land}} + c_{\text{snt}} \cdot A_{\text{snt}} + c_{\text{bnt}} \cdot A_{\text{park}} + C_{\text{overhead}}, \quad (38)$$

where:

- C_{land} is the land acquisition cost,
- $A_{\text{snt}} = \sum_d n_d \cdot (a_{\text{int}}^{(d)} + a_{\text{ter}}^{(d)} + a_{\text{common}}^{(d)})$ is the total above-ground built area (apartments, terraces, corridors, and common spaces),

- c_{snt} is the per- m^2 construction cost for above-ground floors (typically 25–40 UF/ m^2 in Chile, where UF is the inflation-indexed unit of account),
- c_{bnt} is the per- m^2 construction cost for underground levels (typically 35–55 UF/ m^2 , significantly higher due to excavation, retaining walls, and waterproofing),
- C_{overhead} captures architectural and engineering fees, permits, legal costs, sales commissions, and financing costs (typically 15–25% of direct construction cost).

The objective at the building level is to maximize the developer’s profit—total revenue minus total cost:

$$\max \quad R_{\text{total}} - C_{\text{total}}. \quad (39)$$

6 Computational Complexity

A key advantage of the decomposition approach is that every stage has polynomial (or better) complexity, and the constants involved are small enough for sub-second execution on typical inputs. We analyze the worst-case complexity of each stage for a lot with $|G|$ grid cells, Z_{max} floors, and $|\mathcal{T}_{\text{all}}|$ apartment templates.

Stage	Operation	Complexity
A	Facade setback search	$O(N_{d_f})$ candidates
A	Height map (per candidate)	$O(G \cdot \mathcal{R} \cdot K)$
A	Floor footprint extraction	$O(G \cdot Z_{\text{max}})$
B.1	Skeleton graph construction	$O(G)$
B.1	Corridor enumeration	$O(2^{ E })$, typically $O(1)$
B.2	Strip decomposition	$O(E(\mathcal{C}))$ per corridor
B.3	Knapsack DP per strip	$O(\mathcal{T}_S \cdot W)$
B.4	Joint floor optimization	$O(\mathcal{C}^* \cdot \prod_v \mathcal{O}_v \cdot \mathcal{S} \cdot \mathcal{T}_S \cdot W)$
C	Multi-floor assembly	$O(\mathcal{D})$
C	Parking optimization	$O(\log(1/\epsilon))$ (binary search)
Total (typical)		$O(N_{d_f} \cdot (G \cdot \mathcal{R} \cdot K + \mathcal{D} \cdot \mathcal{T} \cdot W))$

Table 2: Computational complexity by stage. The outer loop over facade setback candidates N_{d_f} multiplies the cost of Stages A–C. Within each candidate, the height map computation (Stage A) and the knapsack DP (Stage B.3) dominate the runtime.

Proposition 1 (Practical Runtime). *For a typical urban lot ($|G| \leq 10\,000$ cells, $Z_{\text{max}} \leq 15$ floors, $|\mathcal{T}_{\text{all}}| \leq 60$ templates, $W \leq 240$ half-meter units, $|\mathcal{R}| \leq 10$ regulations, $K \leq 20$ boundary segments, $|\mathcal{D}| \leq 5$ distinct footprints, $N_{d_f} \leq 25$ facade setback candidates), the computation per facade candidate is:*

- *Stage A:* $10\,000 \times 10 \times 20 = 2 \times 10^6$ operations (height map) plus $10\,000 \times 15 = 1.5 \times 10^5$ operations (footprint extraction),
- *Stage B (per footprint):* at most $45 \times 60 \times 240 = 6.5 \times 10^5$ operations (enumeration $\times$ knapsack), for 5 footprints totaling 3.2×10^6 operations,
- *Stage C:* negligible (a few dozen arithmetic operations).

Per candidate: approximately 5×10^6 operations. Over 25 candidates: 1.25×10^8 operations total, still executing in well under one second on modern hardware. This enables batch evaluation of thousands of lot combinations for portfolio-level investment analysis.

Remark 3 (Comparison with MIP Approach). *A direct mixed-integer programming formulation of the full problem—placing apartment templates on a 2D grid across all floors simultaneously—would require $O(|G| \cdot Z_{\text{max}} \cdot |\mathcal{T}_{\text{all}}|)$ binary variables. For the parameter values above, this yields*

$10\,000 \times 15 \times 60 = 9 \times 10^6$ binary variables, a problem size that is well beyond the practical reach of commercial MIP solvers for exact solutions (and even heuristic solutions would require minutes to hours).

The decomposition approach replaces this monolithic problem with $O(|\mathcal{D}|)$ one-dimensional knapsack instances, each with $O(|\mathcal{T}_S| \cdot W)$ states, yielding a speedup of several orders of magnitude. The decomposition preserves exact optimality within each stage and near-optimality across stages (the only approximation is the Lagrangian relaxation for mix constraints, which converges to within 1–2% of the joint optimum in practice).

7 Conclusion

We have presented a decomposition-based optimization framework for automated building design on irregular urban lots. The framework addresses the full scope of the real estate feasibility problem: from raw lot geometry and zoning regulations to a complete building design with apartment layouts, parking dimensioning, and economic valuation.

The key contributions are:

1. **Height map formulation:** All three-dimensional geometric regulations (setbacks, setbacks, shadows, height limits) are reduced to a two-dimensional discrete function $H(i, j)$, eliminating the need for non-convex continuous optimization of the building envelope. Each regulation is encoded independently as a height bound, and the final envelope is obtained by a simple pointwise minimum. This formulation is both computationally efficient and extensible: adding a new regulatory constraint requires only implementing its height bound function.
2. **Skeleton-based corridor routing:** The medial axis of the floor footprint provides a natural and compact representation of valid corridor layouts. By constructing a skeleton graph and enumerating its connected subtrees, we systematically explore all structurally feasible corridor topologies. This approach handles L-shaped, H-shaped, U-shaped, and T-shaped floor plans within a unified framework, without requiring shape-specific heuristics or case analysis.
3. **Strip knapsack decomposition:** Once the corridor is fixed, the two-dimensional apartment layout problem decomposes into independent one-dimensional bounded knapsack problems—one per strip of usable space alongside the corridor. Each knapsack is solved exactly by dynamic programming in pseudo-polynomial time, providing global optimality within each strip. The key insight is that corridor-based residential buildings have an inherent guillotine-cut structure: the corridor is a horizontal cut across the floor, and apartment partition walls are vertical cuts within each strip.
4. **Revenue-driven optimization:** By incorporating market pricing, orientation quality, and apartment type preferences directly into the template revenues, the framework optimizes economic value rather than geometric area. The Lagrangian relaxation mechanism for market mix constraints ensures that the building produces a balanced apartment distribution aligned with market demand, while preserving the separability of the strip-level knapsack decomposition.
5. **Scalability:** The full building optimization—from lot polygon to detailed floor plans and parking design—executes in sub-second time on a single processor core. This enables applications that would be infeasible with traditional approaches: batch evaluation of thousands of lot combinations in a real estate portfolio, sensitivity analysis across regulatory variants and market scenarios, and real-time exploration of design alternatives during feasibility meetings with stakeholders.

Limitations and extensions. The framework assumes that optimal floor plans follow a corridor-based typology—double-loaded or single-loaded corridors with rectangular apartments opening onto them. This assumption holds for the vast majority of mid-rise residential buildings (4–15 floors) in urban settings, which account for the bulk of new housing supply in Chilean cities. However, the framework does not cover:

- **Point towers:** tall, slender buildings (typically 20+ floors) where apartments radiate from a central core rather than aligning along a corridor. These require a different decomposition based on angular partitioning around the core.
- **Free-form architecture:** buildings with curved walls, non-orthogonal geometries, or irregular apartment shapes. While these are rare in mass-market residential development, they appear in high-end or signature projects.
- **Mixed-use buildings:** buildings with commercial or office floors at lower levels and residential floors above. The framework handles the residential portion, but the commercial floor layout would require a different optimization model.

Extensions to handle point towers could incorporate pie-shaped or trapezoidal apartment templates arranged radially, solved by a circular knapsack variant. Mixed-use buildings could be addressed by optimizing each use type independently and coupling them through shared constraints (parking, circulation core, constructibility). These extensions fit naturally within the same decomposition structure presented here.